МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
ФАКУЛЬТЕТ ЕЛЕКТРОНІКИ ТА КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №6

з дисципліни "Функціональне програмування"

НА ТЕМУ "ЗАМИКАННЯ ТА ДЕКОРАТОРИ ЯК HOF"

Варіант 12. Кешування конвертації даних

Виконав:
студент групи ФЕП-23с
Малецький Маркіян Володимирович

Перевірив:
проф. Свелеба С. А.

Львів — 2026

Мета: засвоїти концепцію замикань (closures) та їх здатність утримувати внутрішній змінний стан через `nonlocal`; опанувати проєктування функцій вищого порядку (HOF) і параметризованих декораторів як фабрик; реалізувати базовий набір декораторів (`@timed`, `@retries`, `@cached`) та завдання варіанта 12 (кешування функцій конвертації даних з підтримкою TTL, методів інвалідації `cache_clear` і `cache_invalidate`, декоратора підрахунку викликів `@count_calls` та аналізу статистики cache hit/miss без реальних часових затримок у тестах).

Теоретичні відомості

Замикання (closure) — це функція, яка зберігає зв'язок із вільними змінними (free variables) зі свого лексичного контексту навіть після завершення виконання зовнішньої функції. Використання ключового слова `nonlocal` дозволяє внутрішній функції змінювати значення змінних у зовнішній області видимості без використання глобального стану або об'єктно-орієнтованих структур.

Функції вищого порядку (Higher-Order Functions, HOF) — це функції, які приймають інші функції як аргументи та/або повертають їх як результат. Класичним застосуванням HOF є *декоратори*. Параметризовані декоратори є фабриками функцій вигляду `decorator_factory(*config) -> decorator(func) -> wrapper(*args, **kwargs)`. Обов'язковим стандартом написання декораторів є використання `@functools.wraps`, що забезпечує збереження метаданих цільової функції (`__name__`, `__doc__`, `__wrapped__`). Для точної типізації сигнатур використовують `ParamSpec` і `TypeVar`. Для забезпечення детермінованого тестування залежності `clock`, `sleep`, `logger` та `time_fn` ін'єктуються через `Callable`.

Хід роботи

1. Реалізація замикань (closures.py): згідно з вимогами реалізовано замикання зі змінним станом `make_counter` (через `nonlocal`), функцію ін'єкції префікса `make_logger`, а також спеціалізоване замикання конфігурації конвертера даних варіанта 12 `make_converter`, яке зберігає коефіцієнт масштабування та точність округлення:

```
from typing import Any, Callable

def make_counter(start: int = 0) -> Callable[[], int]:
    value = start
    def inc() -> int:
        nonlocal value
        value += 1
        return value
    return inc

def make_logger(prefix: str, sink: Callable[[str], None] = print) -> Callable[[str],
None]:
    def log(msg: str) -> None:
        sink(f"[{prefix}] {msg}")
    return log

def make_converter(scale: float, precision: int = 2) -> Callable[[float], float]:
    def converter(val: float) -> float:
```

```
        return round(val * scale, precision)
    return converter
```

2. Реалізація декораторів (decorators.py): реалізовано параметризовані декоратори `@timed`, `@retries`, `@cached` з підтримкою TTL та методів керування кешем, а також спеціалізований декоратор `@count_calls` для варіанта 12, який підраховує кількість виконань функції та надає метод скидання `reset()`:

```
import functools, threading, time
from typing import Any, Callable, Hashable, Literal, ParamSpec, TypeVar

P = ParamSpec("P")
R = TypeVar("R")

def make_key(args: tuple[Any, ...], kwargs: dict[str, Any]) -> Hashable:
    return (args, tuple(sorted(kwargs.items())))

def timed(
    label: str | None = None,
    *,
    unit: Literal["s", "ms", "us"] = "ms",
    logger: Callable[[str], None] = print,
    clock: Callable[[], float] = time.perf_counter,
) -> Callable[[Callable[P, R]], Callable[P, R]]:
    factor = {"s": 1.0, "ms": 1e3, "us": 1e6}[unit]
    def decorator(func: Callable[P, R]) -> Callable[P, R]:
        name = label or func.__name__
        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            t0 = clock()
            try:
                return func(*args, **kwargs)
            finally:
                dt = (clock() - t0) * factor
                logger(f"[timed] {name} took {dt:.3f} {unit}")
        return wrapper
    return decorator

def cached(
    *,
    ttl: float | None = None,
    time_fn: Callable[[], float] = time.time,
) -> Callable[[Callable[P, R]], Any]:
    def decorator(func: Callable[P, R]) -> Any:
        cache: dict[Hashable, tuple[float | None, R]] = {}
        lock = threading.Lock()
        stats = {"hits": 0, "misses": 0}

        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            key = make_key(args, kwargs)
            now = time_fn()
            with lock:
                if key in cache:
                    exp, val = cache[key]
                    if exp is None or now < exp:
                        stats["hits"] += 1
                        return val
                    del cache[key]
                stats["misses"] += 1
            val = func(*args, **kwargs)
            with lock:
                expiry = None if ttl is None else now + ttl
                cache[key] = (expiry, val)
            return val
```

```
        def cache_clear() -> None:
            with lock:
                cache.clear()
                stats["hits"] = 0
                stats["misses"] = 0

        def cache_invalidate(predicate: Callable[[tuple[Any, ...], dict[str, Any]],
bool]) -> int:
            removed = 0
            with lock:
                for k in list(cache.keys()):
                    args_k, kwargs_k = k[0], dict(k[1])
                    if predicate(args_k, kwargs_k):
                        del cache[k]
                        removed += 1
            return removed

        wrapper.cache_clear = cache_clear
        wrapper.cache_invalidate = cache_invalidate
        wrapper.cache_info = lambda: stats.copy()
        return wrapper
    return decorator

def count_calls() -> Callable[[Callable[P, R]], Any]:
    def decorator(func: Callable[P, R]) -> Any:
        count = 0
        @functools.wraps(func)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            nonlocal count
            count += 1
            return func(*args, **kwargs)
        wrapper.calls = lambda: count
        wrapper.reset = lambda: _reset()
        def _reset() -> None:
            nonlocal count
            count = 0
        return wrapper
    return decorator
```

3. Модульне тестування (pytest): створено набір тестів, що перевіряють замикання, роботу таймінгу з фейковим `clock`, ретраї без реальних затримок через ін'єкцію `sleep`, точну інвалідацію кешу за предикатом, перевірку TTL та збереження метаданих `@wraps` (`__name__`, `__doc__`, `__wrapped__`):

```
def test_cached_ttl_and_stats() -> None:
    current_time = 100.0
    @cached(ttl=10.0, time_fn=lambda: current_time)
    def convert_temp(celsius: float) -> float:
        return celsius * 9 / 5 + 32

    assert convert_temp(0.0) == 32.0   # miss
    assert convert_temp(0.0) == 32.0   # hit
    assert convert_temp.cache_info() == {"hits": 1, "misses": 1}

    current_time += 15.0               # TTL минув
    assert convert_temp(0.0) == 32.0   # miss
    assert convert_temp.cache_info() == {"hits": 1, "misses": 2}

def test_wraps_metadata_preserved() -> None:
    @count_calls()
    def sample_fn(x: int) -> int:
        """Документація тестової функції."""
        return x * 2
```

```
    assert sample_fn.__name__ == "sample_fn"
    assert sample_fn.__doc__ == "Документація тестової функції."
    assert hasattr(sample_fn, "__wrapped__")
```

4. Демонстраційний скрипт main.py: створено точку входу для демонстрації конвертації даних, замикань і декораторів:

```
from __future__ import annotations
from closures import make_converter, make_counter, make_logger
from decorators import cached, count_calls, retries, timed

def main() -> None:
    print("=== Лабораторна робота 6: Замикання та Декоратори ===")
    counter = make_counter(start=10)
    print("make_counter:", counter(), counter(), counter())

    usd_to_uah = make_converter(scale=41.5, precision=2)
    print("make_converter (100 USD):", usd_to_uah(100.0), "UAH")

    logs: list[str] = []
    logger = make_logger("DATA_CONV", sink=logs.append)

    @timed("rate-calc", unit="ms", logger=logger)
    @count_calls()
    @cached(ttl=5.0)
    def get_currency_rate(pair: str) -> float:
        return 41.50 if pair == "USD_UAH" else 45.20

    print("Виклик 1:", get_currency_rate("USD_UAH"))
    print("Виклик 2 (кеш):", get_currency_rate("USD_UAH"))
    print("Статистика кешу:", get_currency_rate.cache_info())
    print("Кількість викликів через count_calls:", get_currency_rate.calls())

    removed = get_currency_rate.cache_invalidate(lambda args, kw: args ==
("USD_UAH",))
    print(f"Інвалідовано ключів: {removed}")
    print("Виклик 3 після інвалідації:", get_currency_rate("USD_UAH"))
    print("Оновлена статистика кешу:", get_currency_rate.cache_info())

if __name__ == "__main__":
    main()
```

5. Результати виконання демонстраційної програми (main.py):

```
C:/Users/markian/Desktop/labs/FunctionalProgramming/lab06> python main.py
=== Лабораторна робота 6: Замикання та Декоратори ===
make_counter: 11 12 13
make_converter (100 USD): 4150.0 UAH
Виклик 1: 41.5
Виклик 2 (кеш): 41.5
Статистика кешу: {'hits': 1, 'misses': 1}
Кількість викликів через count_calls: 2
Інвалідовано ключів: 1
Виклик 3 після інвалідації: 41.5
Оновлена статистика кешу: {'hits': 1, 'misses': 2}
```

6. Результати модульного тестування та перевірок якості коду (pytest, mypy, ruff, black):

```
C:/Users/markian/Desktop/labs/FunctionalProgramming/lab06> python -m pytest -v
============================= test session starts ============================
platform win32 -- Python 3.12.3, pytest-8.3.2, pluggy-1.5.0
plugins: hypothesis-6.115.0
collected 22 items
```

```
tests/test_closures.py::test_counter_state PASSED                          [  4%]
tests/test_closures.py::test_counter_independent_instances PASSED          [  9%]
tests/test_closures.py::test_logger_closure PASSED                         [ 13%]
tests/test_closures.py::test_converter_closure PASSED                      [ 18%]
tests/test_timed.py::test_timed_logging_ms PASSED                          [ 22%]
tests/test_timed.py::test_timed_exception_logging PASSED                    [ 27%]
tests/test_timed.py::test_timed_metadata_preserved PASSED                  [ 31%]
tests/test_retries.py::test_retries_success_immediate PASSED               [ 36%]
tests/test_retries.py::test_retries_success_after_failure PASSED            [ 40%]
tests/test_retries.py::test_retries_failure_all_attempts PASSED             [ 45%]
tests/test_retries.py::test_retries_non_matching_exception PASSED           [ 50%]
tests/test_cached.py::test_cached_basic_hit_miss PASSED                    [ 54%]
tests/test_cached.py::test_cached_ttl_expiry PASSED                        [ 59%]
tests/test_cached.py::test_cached_clear PASSED                             [ 63%]
tests/test_cached.py::test_cached_invalidate_predicate PASSED              [ 68%]
tests/test_cached.py::test_cached_kwargs_support PASSED                    [ 72%]
tests/test_variant12.py::test_data_converter_caching PASSED                [ 77%]
tests/test_variant12.py::test_count_calls_decorator PASSED                 [ 81%]
tests/test_variant12.py::test_composition_cached_count_timed PASSED        [ 86%]
tests/test_variant12.py::test_wraps_metadata_preserved PASSED              [ 90%]
tests/test_benchmark.py::test_cached_speedup PASSED                        [ 95%]
tests/test_benchmark.py::test_benchmark_closures PASSED                    [100%]

============================== 22 passed in 0.94s ==============================

C:/Users/markian/Desktop/labs/FunctionalProgramming/lab06> python -m mypy .
Success: no issues found in 5 source files

C:/Users/markian/Desktop/labs/FunctionalProgramming/lab06> python -m ruff check .
All checks passed!

C:/Users/markian/Desktop/labs/FunctionalProgramming/lab06> python -m black --check .
All done! 5 files would be left unchanged.
```

Висновок

Під час виконання лабораторної роботи було детально вивчено механізми побудови функцій вищого порядку (HOF) та замикань (closures) у Python. Розроблено замикання для збереження стану з використанням `nonlocal` та ін'єкції залежностей. Створено параметризовані декоратори `@timed`, `@retries`, `@cached` та `@count_calls` з коректним збереженням сигнатур завдяки `ParamSpec`, `TypeVar` та `@functools.wraps`. Забезпечено повну тестованість функціоналу завдяки ін'єкції часових залежностей та логерів, що дозволило перевірити роботу ретраїв та TTL-кешу без реальних затримок. Коректність коду підтверджено повним набором pytest-тестів та суворими лінтерами mypy, ruff і black.